

dwes

CASO 3

O MISTÉRIO DO CONFIGMAP QUE NÃO ATUALIZA



LINUXTIPS

O CASO: A CONFIGURAÇÃO FANTASMA



E para fechar nosso capítulo sobre Kubernetes, um mistério que já fez muito desenvolvedor arrancar os cabelos. Você tem sua aplicação configurada para ler seus parâmetros de um ConfigMap, o objeto do Kubernetes para armazenar configurações não sensíveis. Você precisa mudar o valor de uma configuração, por exemplo, o nível de log da sua aplicação de **INFO** para **DEBUG**.

Você edita o ConfigMap com **kubectl edit configmap minha-app-config**, salva a alteração, e espera que sua aplicação comece a cuspir logs de **debug**. Mas... nada. Silêncio. A aplicação continua rodando com a configuração antiga. Você entra no Pod com **kubectl exec** e verifica o arquivo de configuração que deveria ter sido montado a partir do ConfigMap, e ele ainda está com o valor antigo. É como se a sua alteração tivesse sido ignorada, e o Pod estivesse vivendo no passado, com uma configuração fantasma.

Sim ou não, turma, que essa é uma situação frustrante? Você mudou a fonte da verdade (o ConfigMap), mas os Pods se recusam a enxergar a nova realidade.



A INVESTIGAÇÃO: COMO O KUBERNETES LIDA COM MUDANÇAS?

Para resolver esse mistério, precisamos entender como o Kubernetes propaga as alterações de ConfigMaps (e Secrets) para os Pods. E a resposta é: depende de como você está usando o ConfigMap.



PISTA 1: VARIÁVEIS DE AMBIENTE VS. VOLUMES MONTADOS

Existem duas maneiras principais de um Pod consumir um ConfigMap:

Como Variáveis de Ambiente

Como um Volume Montado

E aqui está o pulo do gato, a pista crucial: **as variáveis de ambiente de um container são definidas apenas quando o container é criado e nunca são atualizadas.** Se você usa um ConfigMap para injetar variáveis de ambiente, qualquer alteração que você fizer no ConfigMap não será refletida nos Pods que já estão rodando.

Por outro lado, se você monta o ConfigMap como um volume, o Kubernetes é mais esperto. O Kubelet em cada nó periodicamente verifica se os ConfigMaps montados foram atualizados. Se sim, ele atualiza os arquivos dentro do volume montado no Pod. Esse processo não é instantâneo, pode levar até um minuto (o período de sincronização do Kubelet), mas acontece!

Então, a nossa primeira hipótese é: estamos usando o ConfigMap como variáveis de ambiente.

PISTA 2: A APLICAÇÃO PRECISA SER REINICIADA?

Ok, digamos que estamos usando a abordagem correta, montando o ConfigMap como um volume. Nós esperamos um minuto, entramos no Pod e vemos que o arquivo `/etc/config/meu-parametro` foi, de fato, atualizado. Sucesso! Mas... a aplicação ainda está usando o valor antigo. Por quê?

O problema agora não é mais do Kubernetes, é da sua aplicação. Muitas aplicações leem seus arquivos de configuração apenas uma vez, na inicialização. Elas não ficam "assistindo" o arquivo para ver se ele muda. Se a sua aplicação carregou o valor **INFO** na inicialização, ela vai continuar usando esse valor até que seja reiniciada, mesmo que o arquivo no disco mude para **DEBUG**.



A REVELAÇÃO: A MENTE POR TRÁS DO TROUBLESHOOTING

OBSERVAÇÃO DO SINTOMA:

Alterações em um ConfigMap não são refletidas em uma aplicação rodando em um Pod.

HIPÓTESE 1 (MÉTODO DE CONSUMO):

Verificamos o deployment.yaml para ver como o ConfigMap está sendo consumido. Se for por variáveis de ambiente, entendemos que a atualização não é automática e que os Pods precisam ser recriados.

HIPÓTESE 2 (PROPAGAÇÃO DO VOLUME):

Se o ConfigMap for montado como um volume, verificamos (usando kubectl exec e cat) se o arquivo dentro do Pod foi atualizado após um ou dois minutos.

HIPÓTESE 3 (LÓGICA DA APLICAÇÃO):

Se o arquivo no Pod atualizou, mas a aplicação não, o problema está na aplicação, que não recarrega sua configuração dinamicamente.

A SOLUÇÃO:

Se o problema for o uso de variáveis de ambiente:

A solução é forçar a recriação dos Pods. A maneira mais limpa de fazer isso é com o comando **kubectl rollout restart deployment/minha-app**. Isso vai acionar um rolling update, criando novos Pods que lerão a nova versão do ConfigMap na sua inicialização.

Se o problema for a aplicação que não recarrega a configuração:

A solução ideal é modificar a aplicação para que ela possa recarregar sua configuração sem precisar reiniciar. Se isso não for possível, a solução é a mesma do ponto anterior: forçar um **rollout restart** do Deployment sempre que a configuração mudar.



LIÇÕES DO CASO

Este mistério nos ensina que a integração entre o Kubernetes e a nossa aplicação é uma via de mão dupla. Não basta o Kubernetes fazer a sua parte (atualizar o volume), a aplicação também precisa fazer a dela (ler a atualização).

Entender as nuances de como os ConfigMaps são propagados é crucial. A regra de ouro é: **prefira montar ConfigMaps como volumes em vez de usá-los para variáveis de ambiente**, pois isso te dá a possibilidade de atualizações dinâmicas. E, se possível, construa aplicações que sejam inteligentes o suficiente para detectar e recarregar suas configurações quando elas mudam.

